Programmazione 2

a.a. 2023-2024

funzioni ricorsive su liste

Richiami su algoritmi ricorsivi

Algoritmo ricorsivo

Procedimento che utilizza se stesso per fare un calcolo

Esempio: definizione del fattoriale

Dato n, numero naturale, il suo fattoriale ! è calcolato come segue:

```
n! = 1              se n vale 0 oppure 1

n! = n * (n-1)!     se n > 0
```

Algoritmo ricorsivo

Procedimento che utilizza se stesso per fare un calcolo

$n! = 1$ se n vale 0 oppure 1

$n! = n * (n-1)!$ se $n > 0$

$5! = 5 * 4! =$

$= 5 * 4 * 3!$

$= 5 * 4 * 3 * 2!$

$= 5 * 4 * 3 * 2 * 1!$

5! è calcolato come 5 per il fattoriale del numero che precede 5 stesso

In C

```
int fattoriale (int n)
{
    if (n<0) fprintf(stderr, “errore”);
    else if (n <= 1) return 1;
   else return n*fattoriale(n-1);
}
```

il fattoriale di un numero è calcolato come il numero per il fattoriale del numero che lo precede

Caso base: punto di stop

```
int fattoriale (int n)
{
    if (n<0) fprintf(stderr, “errore”);
    else if (n <= 1) return 1;
   else return n*fattoriale(n-1);
}
```

Chiamata ricorsiva: invocazione stessa funzione

```
int fattoriale (int n)
{
    if (n<0) fprintf(stderr, “errore”);
    else if (n <= 1) return 1;
   else return n*fattoriale(n-1);
}
```

Ricorsione in coda

```
int fattoriale (int n)
{
    if (n<0) fprintf(stderr, “errore”);
    else if (n <= 1) return 1;
    else return n*fattoriale(n-1);
}
```

dopo la chiamate ricorsiva non si esegue altro codice (return)

Progresso

```
int fattoriale (int n)
{
    if (n<0) fprintf(stderr, “errore”);
    else if (n <= 1) return 1;
    else return n*fattoriale(n-1);
}
```

la nuova chiamata ricorsiva elabora altri dati, non gli stessi di quella attuale

NB: ASSENZA DI PROGRESSO

```
int fattoriale (int n)
{
    if (n<0) fprintf(stderr, “errore”);
    else if (n <= 1) return 1;
    else return n*fattoriale(n);
}
```

ERRORE GRAVE

esecuzione infinita!!!

CONDIZIONE DI CORRETTEZZA

una funzione ricorsiva deve sempre terminare e per farlo occorre essere sicuri che a un certo punto sarà raggiunto il caso base

Ricorsione in testa

```
int fattoriale (int n)
{
    int temp;
    if (n<0) fprintf(stderr, “errore”);
    else if (n > 1) temp = fattoriale(n-1);
    else temp = 1;
    return n*temp;
}
```

occorre come prima operazione, ne seguiranno altre. Viene salvato un valore temporaneo

Caso base

```
int fattoriale (int n)
{
    int temp;
    if (n<0) fprintf(stderr, “errore”);
    else if (n > 1) temp = fattoriale(n-1);
    else temp = 1;
    return n*temp;
}
```

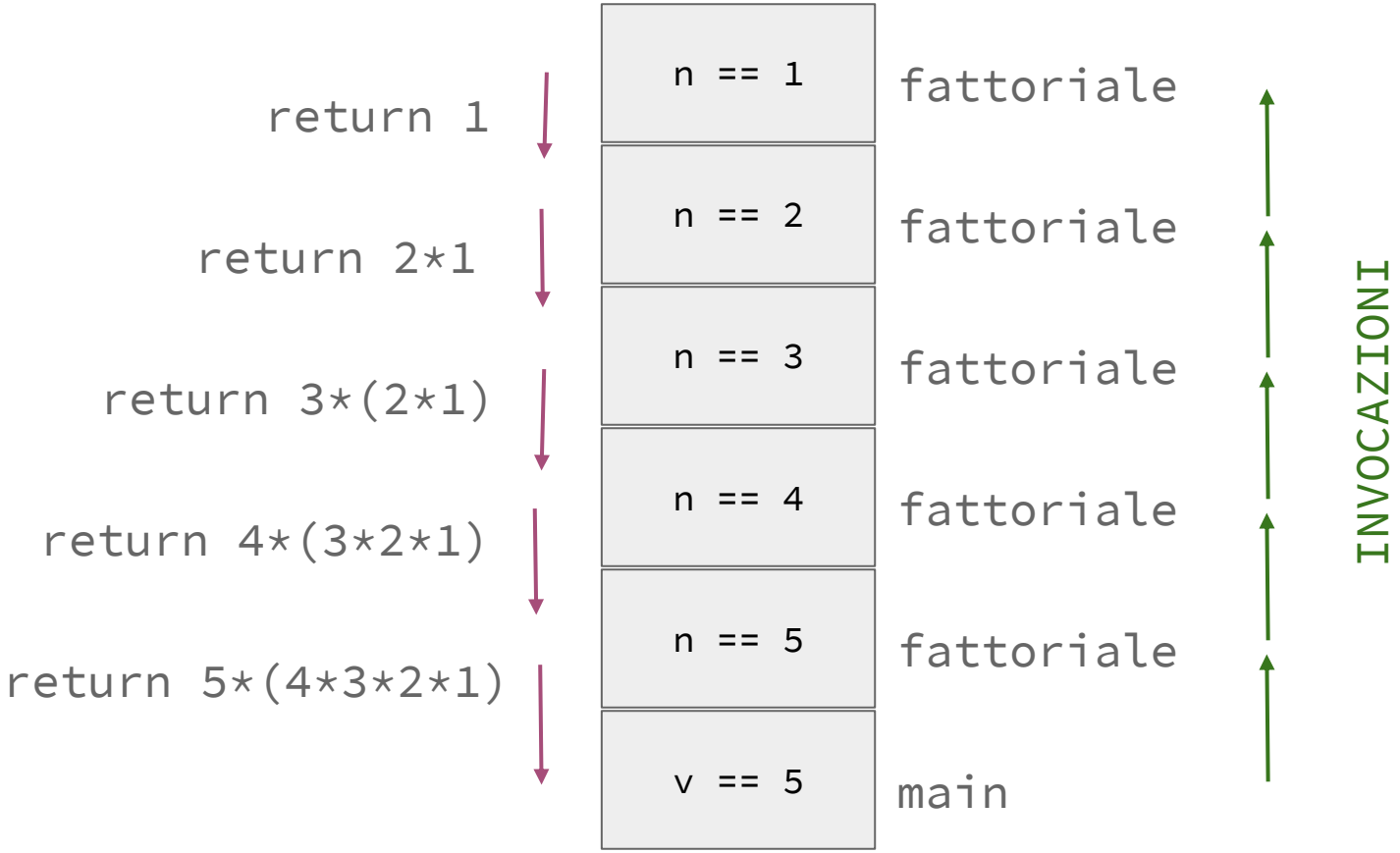
viene raggiunto dopo una serie di chiamate ricorsive

Restituzione

```
int fattoriale (int n)
{
    int temp;
    if (n<0) fprintf(stderr, “errore”);
    else if (n > 1) temp = fattoriale(n-1);
    else temp = 1;
    return n*temp;
}
```

dopo la chiamata ricorsiva si ha ulteriore calcolo

Lo stack di esecuzione



Commenti

- ★ la ricorsione permette di scrivere **codice compatto e facile da interpretare**
- ★ è detta **lineare** quando produce una singola sequenza di chiamate (come nel fattoriale)
- ★ la **ricorsione lineare in coda** corrisponde a un ciclo (segue esempio di soluzione iterativa per il fattoriale)
- ★ la ricorsione comporta un **overhead** (tempo extra di esecuzione e occupazione di memoria) dovuto alla creazione/rimozione del relativo record di attivazione

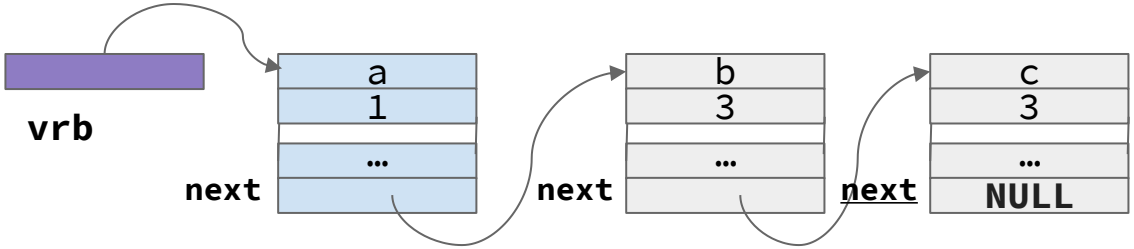
Soluzione iterativa fattoriale

```
int fattoriale (int n)
{
    if (n<=1) return 1;
    else return
    n*fattoriale(n-1);
}
```

```
int fattoriale-iter (int n)
{
    int temp=1;
    for (; n > 1; n--)
    temp=temp*n;
    return temp;
}
```

Ricorsione e liste linkate

Ricorsione e liste linkate



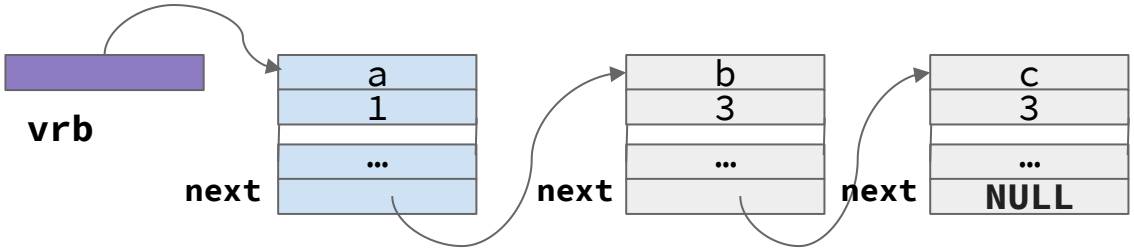
Le liste linkate si prestano a essere manipolate tramite funzioni ricorsive per via della propria **natura ricorsiva**, una lista:

- ★ è vuota
- ★ oppure è un elemento seguito da una lista

Esempio: somma

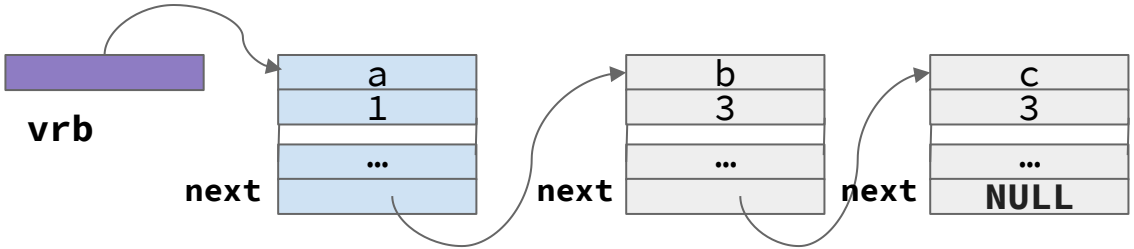
voglio calcolare la somma dei valori contenuti nei nodi di una lista linkata:

- ★ procedimento?
- ★ caso base?
- ★ calcolo?



Esempio: somma

- ★ procedimento?
  percorri la lista accumulando i valori via via incontrati
- ★ caso base?
  lista vuota, anche il caso base produce un valore, quale?
- ★ calcolo?
  somma: val. nodo corrente + somma lista che lo segue



Esempio: somma

calcolo la somma dei valori contenuti nei nodi di una lista linkata

```
int somma-nodi(lista L) {
        if (L == NULL) … LISTA VUOTA, return 0 (elem. neutro)
        else … C'È UN ELEMENTO, CALCOLO E RICORSIONE
}
```

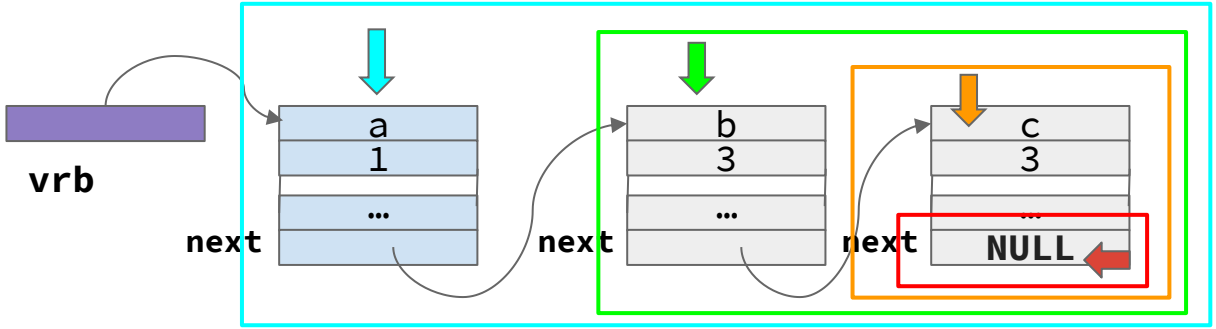
Esempio: somma

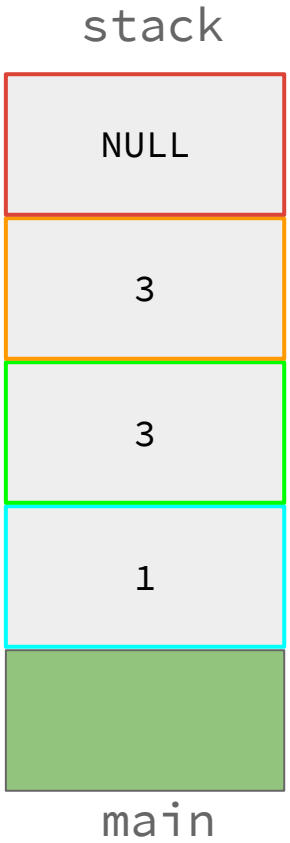
calcolo la somma dei valori contenuti nei nodi di una lista linkata

```
int somma-nodi(lista L) {
      if (L == NULL) return 0;
      else return (L->dato + somma-nodi(L->next));
}
```

Simulazione

```
int somma-nodi(lista L) {
        if (L == NULL) return 0;
        else return L->dato + somma-nodi(L->next);
}
```





RIMOZIONE DI UNA LISTA

Rimozione ricorsiva di una lista

Rimuovere una lista significa non solo togliere degli elementi da essa ma anche **liberare la memoria da essi occupata** tramite free.

```
#include <stdlib.h>
void *malloc(size_t size);
void free(void *ptr);
```

Varie strategie ...

- ★ **rimozione in testa:** iterativa, fare da soli
- ★ **rimozione in coda:** ricorsiva

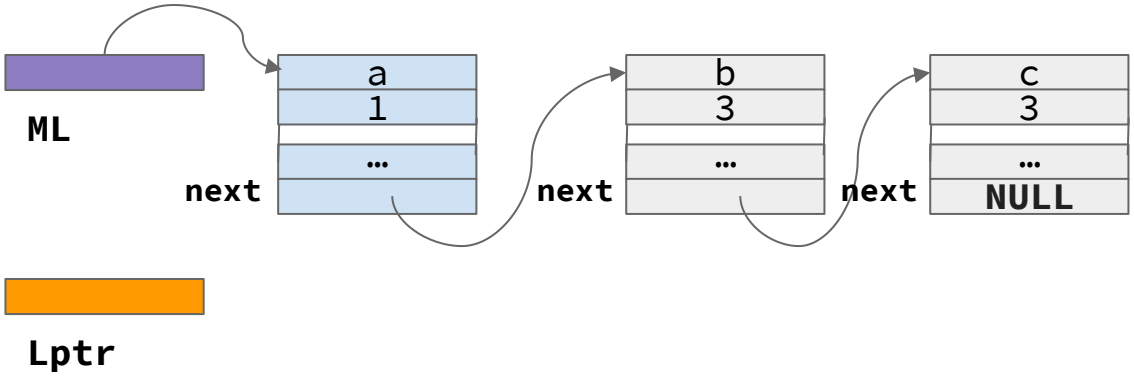
Rimozione in coda con ricorsione in testa

```
/* versione V1 con lista passata per valore */

lista elimina_lista1(lista L) {

    if (L) { // se la lista non è vuota

        L->next = elimina_lista1(L->next);

        free(L);   // libero il nodo (attuale ultimo)

    }

    return NULL;  // segno che dopo non viene più nulla

}
```

Rimozione in coda con ricorsione in testa

```
/* versione V2 con lista passata per riferimento */

void elimina_lista2(lista *Lptr) {

    if (Lptr) // se il parametro è definito

        if (*Lptr) {    // se la lista non è vuota

                elimina_lista2(&((*Lptr)->next)); // mi sposto avanti

                free(*Lptr);   // libero il nodo (attuale ultimo)

                (*Lptr) = NULL;  // segno che dopo non viene più nulla

        }

}
```

Simulazione

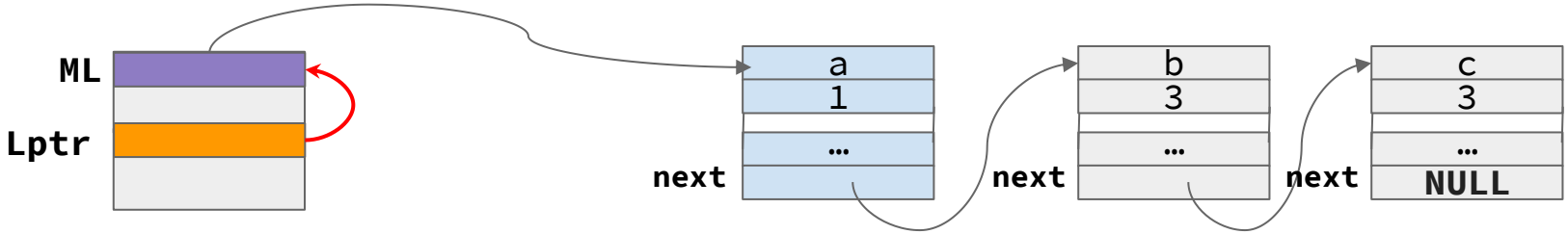


ML: lista nel main (o ambiente chiamante)

Lptr: variabile nella funzione

TARGET: dopo l’esecuzione ML deve valere NULL e tutti i nodi devono essere stati deallocati tramite free

Lista passata per riferimento



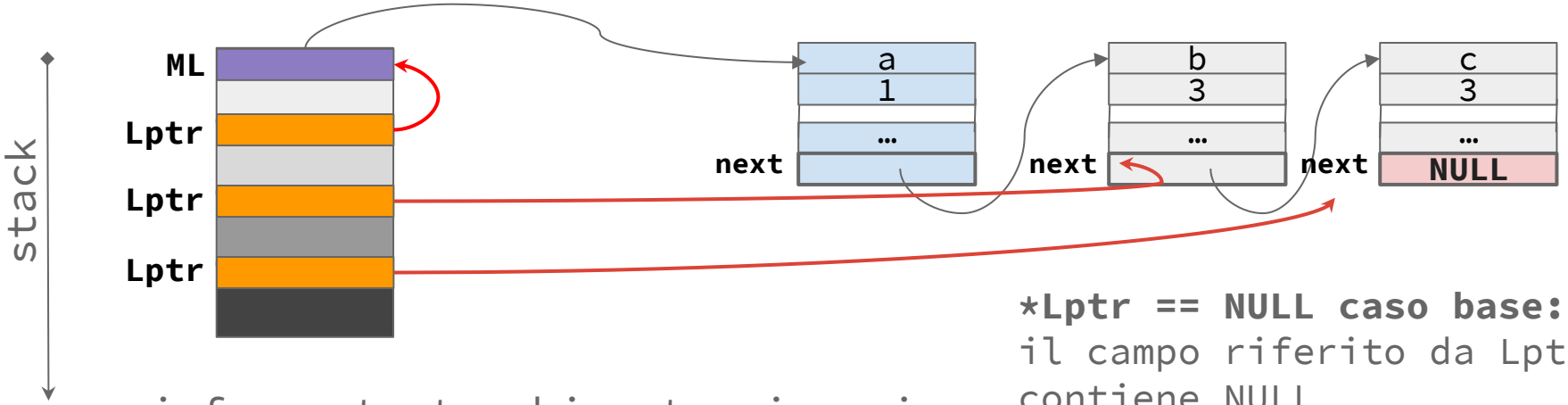
Lptr contiene l'indirizzo di ML

***Lptr** è il contenuto di ML (indirizzo del primo elemento della lista)

(*Lptr)->next è il contenuto del campo next del primo elemento della lista (indirizzo del suo successore)

&((*Lptr)->next) è l'indirizzo di quel campo, permette di modificarlo

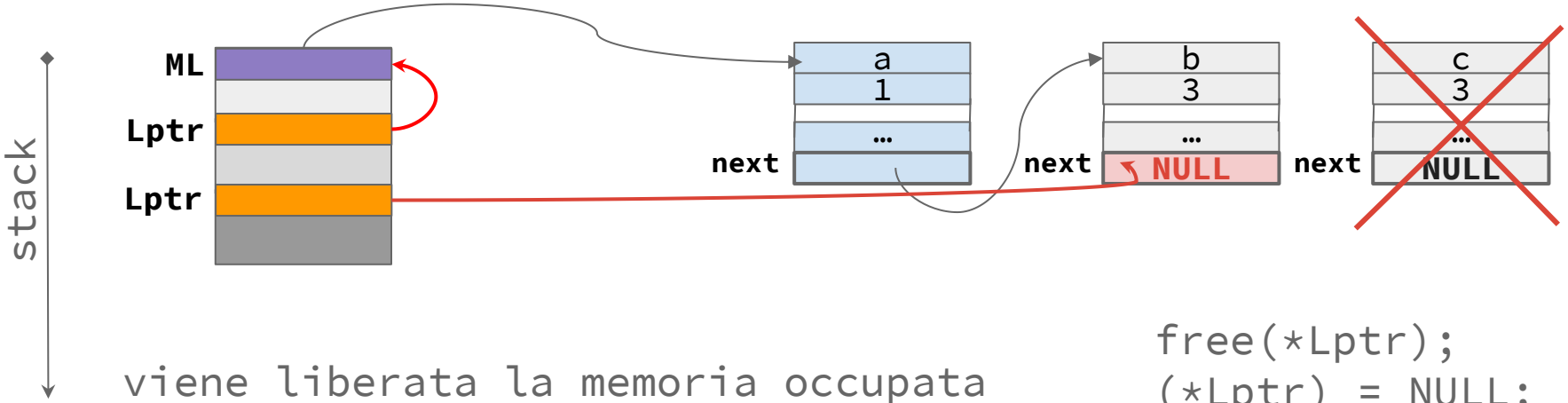
Lista passata per riferimento



***Lptr == NULL caso base:**
il campo riferito da Lptr contiene NULL

si fanno tante chiamate ricorsive fino a raggiungere il NULL, poi via via si chiudono le chiamate ricorsive svolgendo i passi di cancellazione

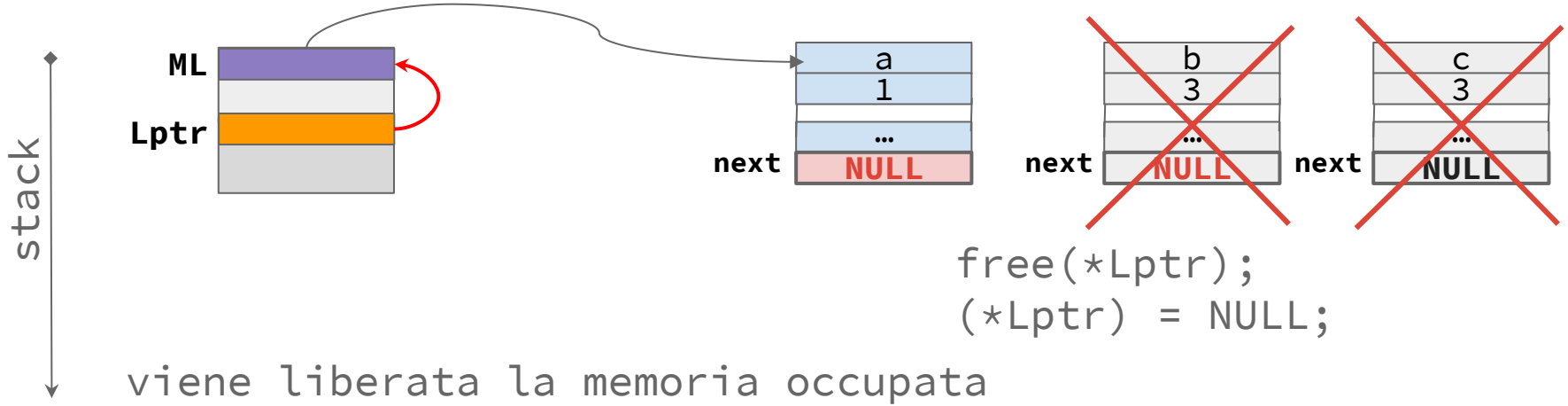
Lista passata per riferimento




```
free(*Lptr);
(*Lptr) = NULL;
```

viene liberata la memoria occupata dall'attuale ultimo elemento della lista e si assegna NULL al next che lo puntava

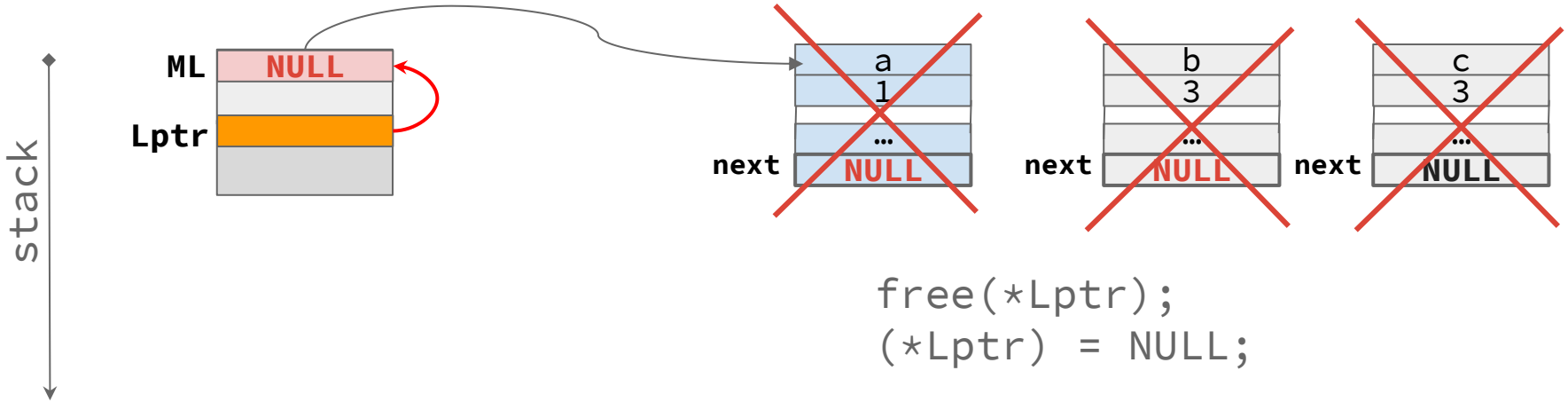
Lista passata per riferimento




```
free(*Lptr);
(*Lptr) = NULL;
```

viene liberata la memoria occupata dall'attuale ultimo elemento della lista e si assegna NULL al next che lo puntava

Lista passata per riferimento




```
free(*Lptr);
(*Lptr) = NULL;
```

infine viene liberata la memoria occupata dall'ultimo elemento rimanente e si assegna NULL alla variabile esterna tramite l'assegnazione del parametro per referenza

Inserzione di un nodo in lista ordinata

Lista linkata: operazioni su elemento generico

- ★ Abbiamo visto funzioni su lista che operano su un elemento in posizione specifica (inserzione/cancellazione in testa/coda)
- ★ Molto spesso occorrono funzioni che operano su elementi **in posizione qualsiasi** nella lista (es. lavorare sugli elementi in posizione dispari)
- ★ Occorre gestire un **maggior numero di casi**
- ★ Caso di studio: **liste ordinate**

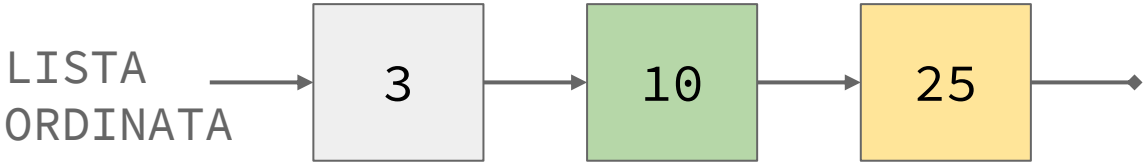
Inserzione in posizione generica

In una lista:

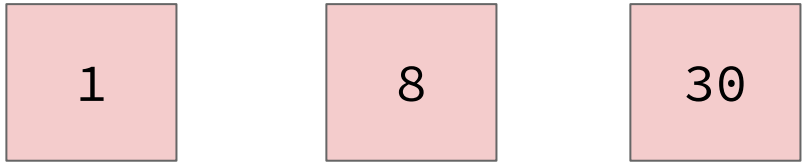
- ★ il primo nodo non ha antecedente
- ★ l’ultimo nodo non ha successore
- ★ tutti gli altri nodi hanno un solo antecedente e un solo successore

Supponiamo che la lista sia **ordinata** per valori crescenti, l’inserzione andrà fatta **trovando prima la posizione giusta**

Esempio

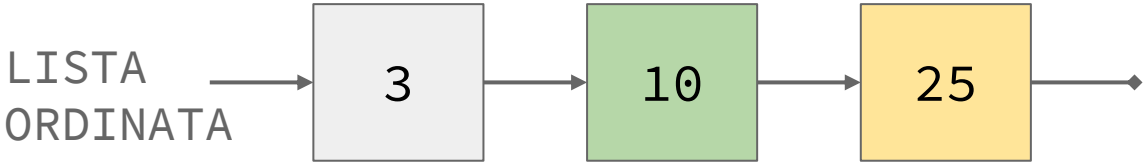


ELEMENTI DA INSERIRE



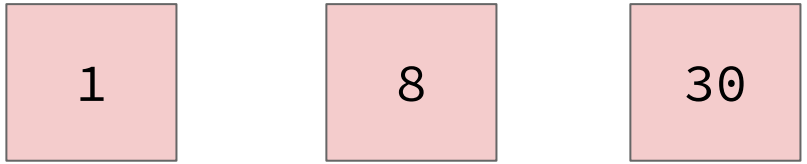
Esempio

PROPRIETÀ



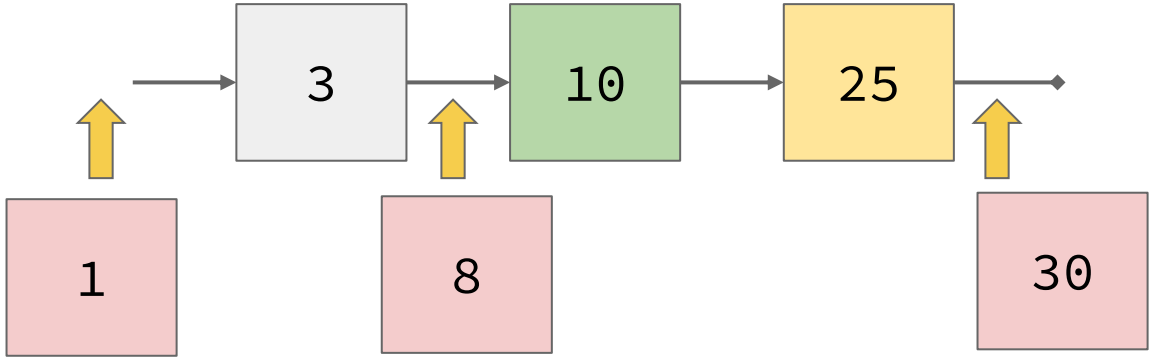
IN PARTENZA:
la lista è
ordinata

ELEMENTI DA INSERIRE

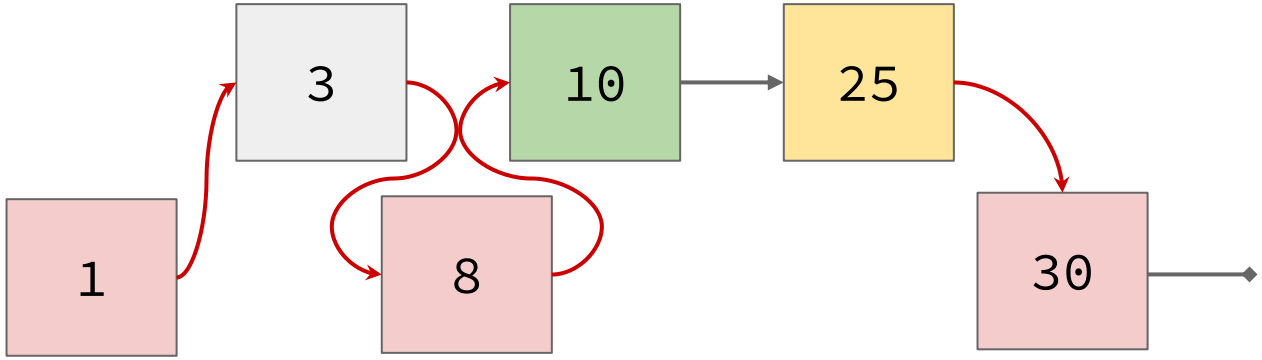


ALLA FINE:
la lista deve
continuare ad
essere ordinata

Esempio



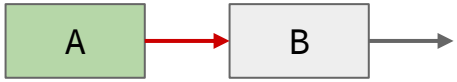
Esempio



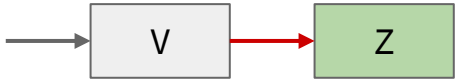
Inserzione in posizione generica

Inserzione:

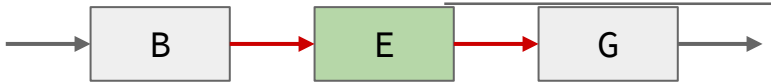
★ <u>in testa</u>: il nodo nuovo avrà come successore l'attuale primo nodo



★ <u>in coda</u>: il nodo nuovo avrà come antecedente l'attuale ultimo nodo



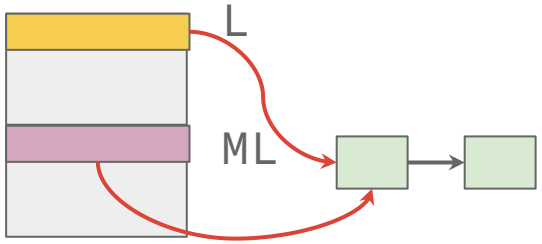
★ altrimenti occorrerà inserirla <u>fra due elementi</u>



Due implementazioni

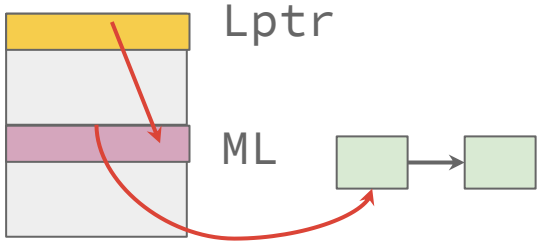
★ **lista passata per valore:**

```
lista ins_ord1(lista L, lista nodo)
```



★ **lista passata per riferimento:**

```
void ins_ord2(lista *Lptr, lista nodo)
```



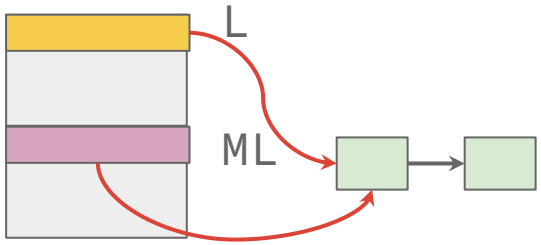
Due implementazioni

- **lista passata per valore:** occorre la collaborazione del chiamante

```
lista ins_ord1(lista L, lista nodo) { … }
```

nel main:

```
lista ML … ;
ML = ins_ord1(ML, crea_nodo( … ));
```



- lista passata per riferimento:

```
void ins_ord2(lista *Lptr, lista nodo)
```

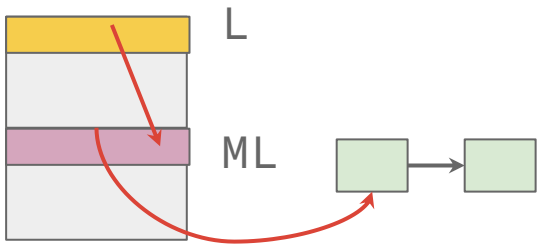
Due implementazioni

- **lista passata per valore:**
  lista ins_ord1(lista L, lista nodo) { … }
- **lista passata per riferimento:** la funzione stessa modifica la variabile esterna

```
void ins_ord2(lista *Lptr, lista nodo) { … }
```

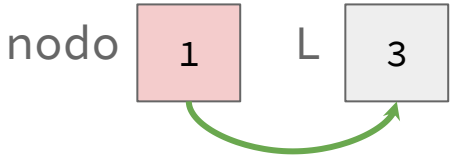
nel main:

```
lista ML … ;
ins_ord2(&ML, crea_nodo( … ));
```



Con passaggio per valore

```
lista ins_ord1(lista L, lista nodo) {
if (L) { // se la lista esiste
        if (nodo->dato <= L->dato) {
                // nodo va inserito in prima posizione
                nodo->next = L;
                return nodo;
        }
        (…continua…)
```

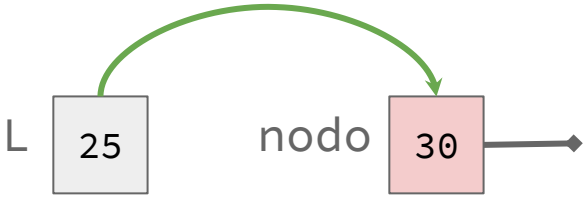


Con passaggio per valore

…

```
else if (!L->next) {
// nodo->dato ha fallito il test precedente quindi
// nodo è maggiore di L, e L non ha successore, è ultimo
// quindi nodo diventa ultimo
    L->next = nodo;
    nodo->next = NULL;
    return L;
}
```

(…continua…)

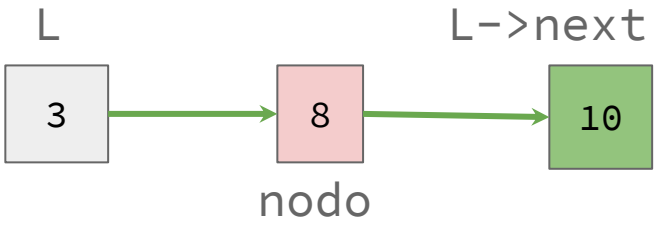


Con passaggio per valore

…

```
else if (nodo->dato < L->next->dato) {
// nodo maggiore di L e minore di L->next
    nodo->next = L->next;
    L->next = nodo;
    return L;
}
```

(…continua…)



Con passaggio per valore

…

```
else {
// nodo maggiore sia di L sia di L-next
// non sono nella posizione giusta, ricorsione
        L->next = ins_ord1(L->next, nodo);
        return L;
}
(…continua…)
```

Con passaggio per valore

```
…
} else return nodo;
// infine se L era NULL, nodo diventa il primo e
// solo elemento della lista
```

Versione con passaggio per riferimento

```
void ins_ord2(lista *Lptr, lista nodo) {

    if (Lptr) // se il parametro ha un valore

       if (*Lptr) // se la lista è non vuota

             if (nodo->dato <= (*Lptr)->dato) {

             // nodo va inserito in prima posizione

                    nodo->next = *Lptr;

                    (*Lptr) = nodo;

             } else if (!((*Lptr)->next)) {

             // L'ultimo è nodo maggiore di L, diventa lui l'ultimo

                    (*Lptr)->next = nodo;

                    nodo->next = NULL;
             }


             (…continua…)
```

Versione con passaggio per riferimento

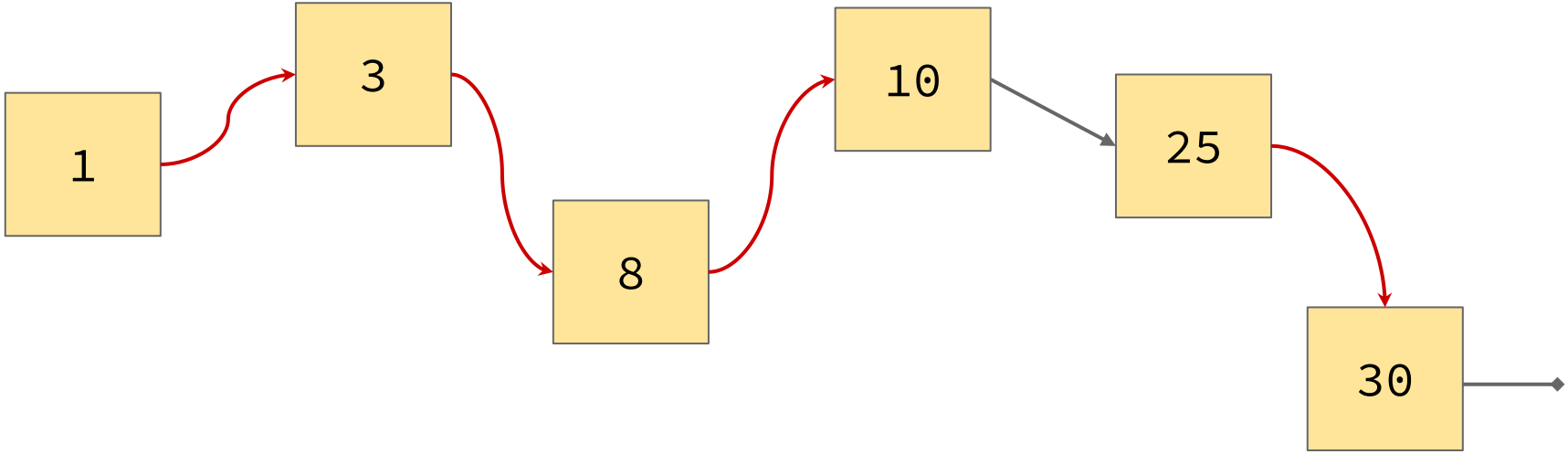
…

```
            else if (nodo->dato < (*Lptr)->next->dato) {
           // nodo maggiore di L e minore di L->next
                   nodo->next = (*Lptr)->next;
                   (*Lptr)->next = nodo;
            } else {
            // nodo maggiore sia di L sia di L-next
                   ins_ord2(&((*Lptr)->next), nodo);
            }
        else (*Lptr) = nodo;
        // *Lptr era NULL e nodo diventa il primo e solo elemento
}
```

Stampa ricorsiva di una lista

Stampa e stampa inversa

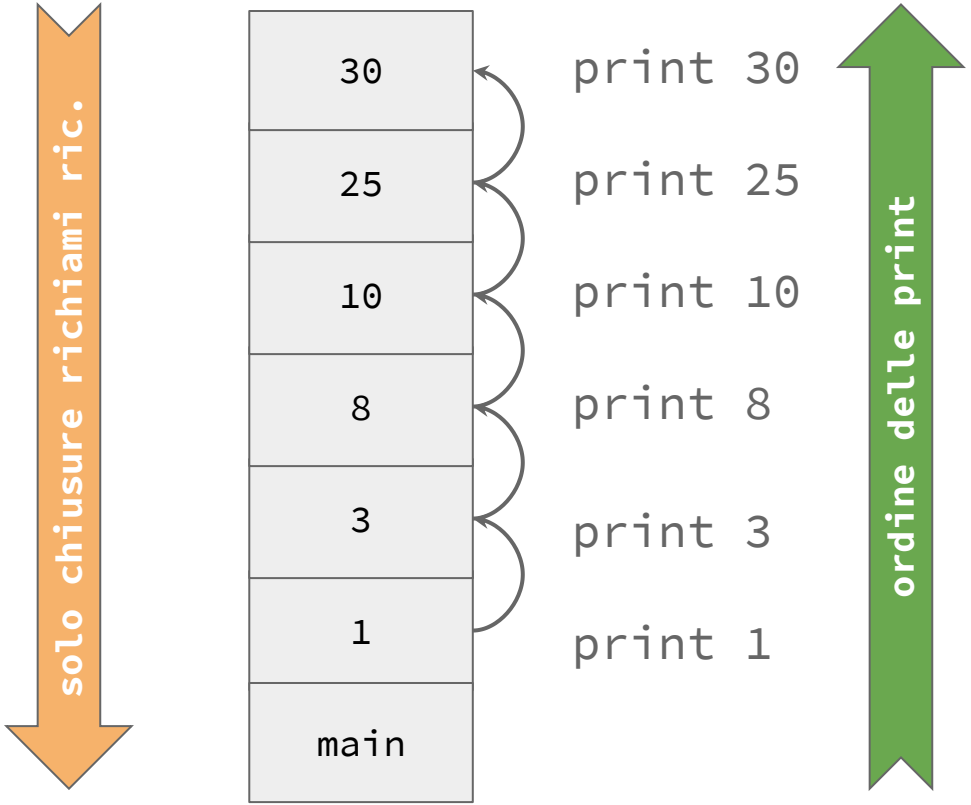
vediamo l'effetto del cambiamento del tipo di ricorsione (in testa vs in coda) sul caso della stampa ricorsiva di una lista



Ricorsione in coda

```
void stampa(lista L) {
    if (L != NULL) {
          printf(" %d,", L->dato); // elaboro nodo
          stampa(L->next);         // ricorsione sul next
    }
    else printf(" /\n");
}
```

Ricorsione in coda: stack di esecuzione



RISULTATO:

1 3 8 10 25 30

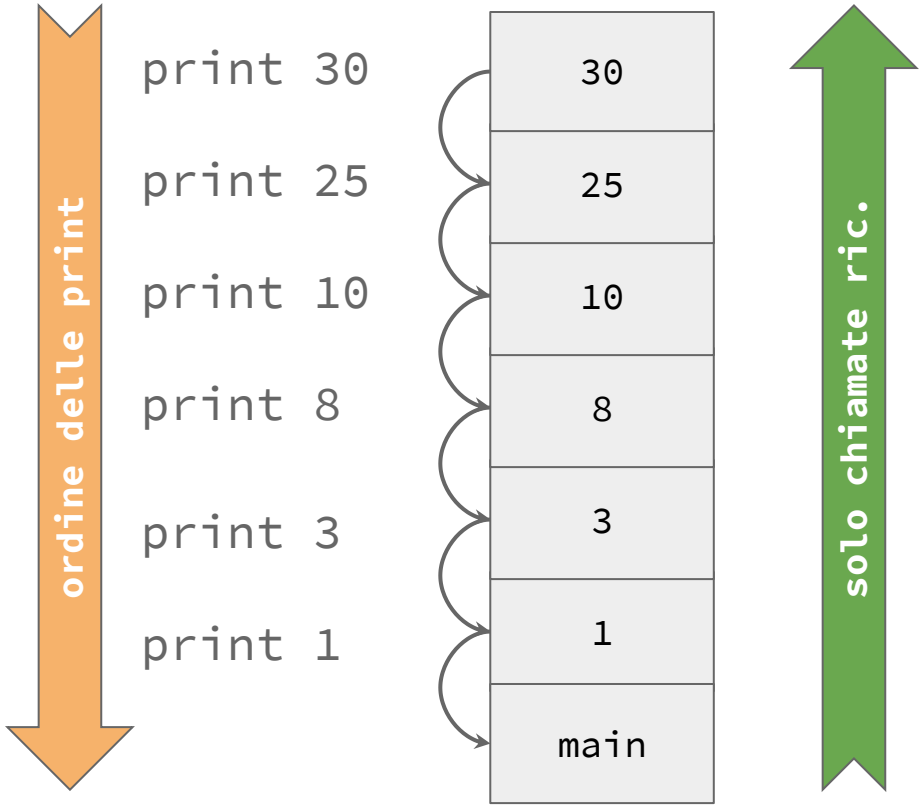
Ricorsione in testa

```
void stampainv(lista L) {
    if (L != NULL) {
        stampainv(L->next);      // ricorsione sul next
        printf(" %d,", L->dato); // elaboro nodo
    }
    else printf(" /\n");
}
```

Ricorsione in testa: stack di esecuzione

RISULTATO:

30 25 10 8 3 1



Errori tipici

Errori tipici

- ★ lista passata per valore: dimenticare di verificare se L vale NULL
- ★ lista passata per riferimento: dimenticare di fare la doppia verifica if (Lptr) if (*Lptr)
- ★ saltare un caso
- ★ non eseguire operazioni nel giusto ordine
- ★ dimenticare di assegnare NULL al next dell'ultimo elemento in coda
- ★ assegnare NULL al next di un elemento da inserire fra due nodi